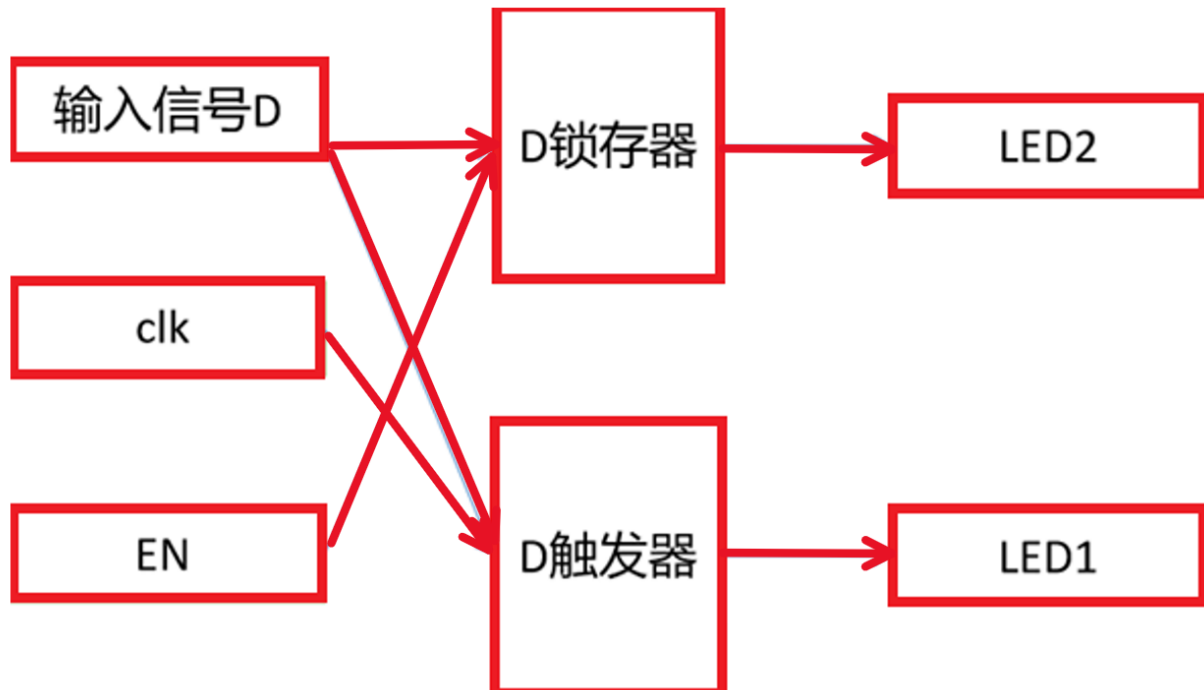


数字逻辑实验报告

实验 1

学 期	2024-2025 学年第 1 学期	实验日期	2024/11/5	
学 院	计算机学院	组 号	学 号	姓 名
专 业	计算机科学与技术（实验班）	15	23071005	侯景祺
班 级	230710			
评 阅 内 容				
任务一	任务二	总结		成 绩
题 目	实验 2: <u>存储器电路设计</u>			
一、实验目的 通过本实验，掌握基本存储电路的分析方法和设计方法；学会使用 Verilog HDL 设计存储器电路。 1.通过触发器、锁存器、寄存器、存储器等时序电路的设计与测试，掌握基本存储电路的分析方法和设计方法； 2.学会使用 Verilog HDL 设计存储器电路。 二、任务一设计与实现 1. 要求 (1) 用行为描述方式实现 D 锁存器，并对该器件进行封装。 (2) 用行为描述方式实现 D 触发器，并对该器件进行封装。 (3) 用图形方式调用 D 锁存器和 D 触发器，两个器件共用同一个输入端 D，并用拨动开关控制，输出端 Q 分别接至 LED 灯，EN 接开关，CLK 接按键，下载后进行功能测试并记录实验现象。 (4) 请编写第 2 条中 D 触发器的 Testbench 测试程序完成仿真。 2. 设计思路 先用行为描述的方式实现 D 锁存器与 D 触发器，分别对其进行封装。在软件种使两个模块共用一个输入端 D，并分配引脚，即可实现功能。（如下图）				



3. 详细设计

如下图，为 2 个封装好的元器件的功能表

D 锁存器功能表

使能	输入	输出	显示
EN	D	Q	LED 灯
1	0	0	亮
1	1	1	灭
0	d	保持上个输出	保持上个状态

D 编译器功能表

时钟信号	输入	输出	显示
CLK	D	Q	LED 灯
↑	0	0	亮
↑	1	1	灭
0	d	保持上个输出	保持上个状态

以下为两个元器件的 Verilog 代码表示，以及 D 触发器的 Testbench 表示，还有总电路的电路图。

```

1 module D_FF(clk, D, Q);
2
3     input clk;
4     input D;
5
6     output Q;
7     reg Q;
8
9     always @ (posedge clk)
10 begin
11     Q<=D;
12 end
13
14 endmodule
15
    
```

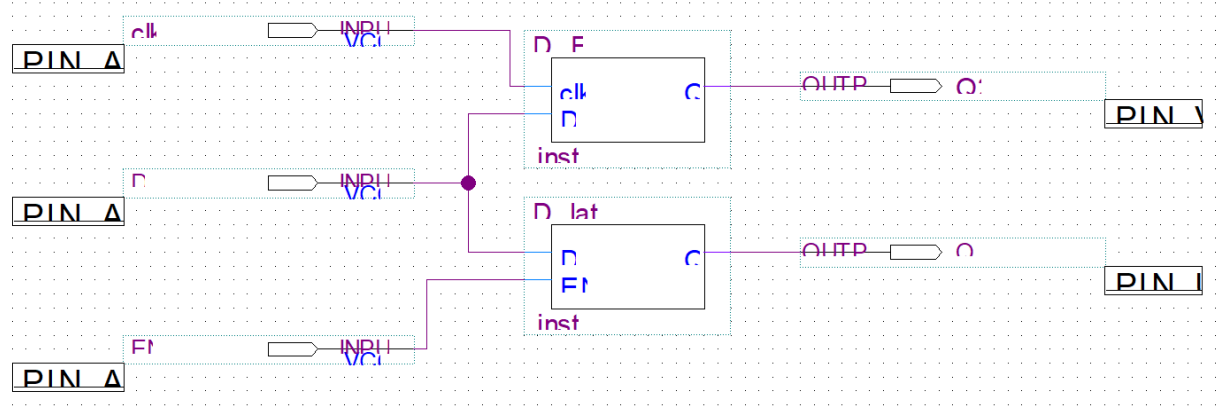
```

1 module D_latch (D, EN, Q);
2
3     input D;
4     input EN;
5
6     output Q;
7     reg Q;
8
9     always @ (D, EN)
10 begin
11     if (EN) Q <= D;
12 end
13
    
```

```

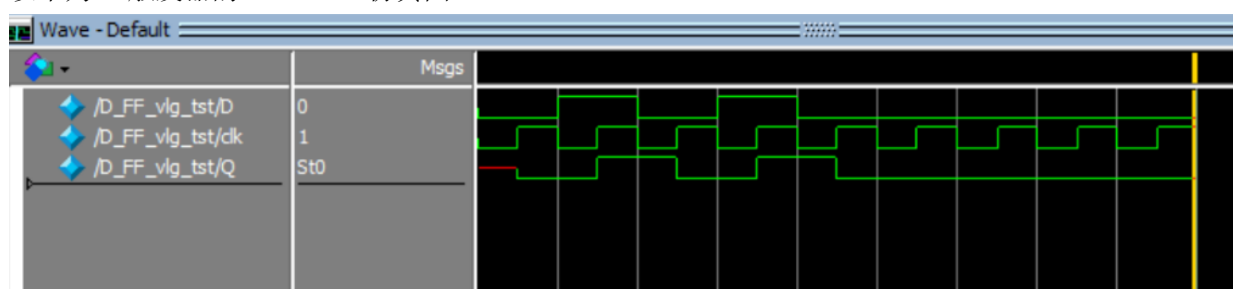
1  `timescale 1ns / 1ps // 设置时间单位和精度
2
3  module D_FF_tb;
4
5      // Testbench 信号声明
6      reg clk;
7      reg D;
8      wire Q;
9
10     // 实例化待测模块 D_FF
11     D_FF uut (
12         .clk(clk),
13         .D(D),
14         .Q(Q)
15     );
16
17     // 生成时钟信号, 每10ns切换一次, 即20ns周期
18     initial begin
19         clk = 0;
20         forever #10 clk = ~clk;
21     end
22
23     // 初始化和激励输入信号
24     initial begin
25         // 打印标题
26         $display("Time\tclk\tD\tQ");
27
28         // 应用输入信号的初始值
29         D = 0;
30         #15; // 等待 15ns, 让 Q 响应时钟的上升沿
31
32         // 改变 D 的值并观察 Q 的响应
33         D = 1; #20; // 在时钟的上升沿时, D 应该传到 Q
34         D = 0; #20; // 另一个时钟上升沿, 观察 D -> Q
35         D = 1; #20;
36         D = 0; #20;
37
38         // 结束仿真
39         #20 $finish;
40     end
41
42     // 打印波形变化
43     initial begin
44         $monitor("%g\t%b\t%b\t%b", $time, clk, D, Q);
45     end
46
47 endmodule
48

```



4. 仿真验证

以下为 D 触发器的 Testbench 仿真图



5. 引脚分配

如下图，D 与 EN 被分配开关，clk 被分配按钮，Q1、Q2 两个输出被分配 LED 灯。

Node Name	Direction	Location	I/O Bank	'REF Group	'O Standar
D	Unknown	PIN_AB17	4	B4_N0	2.5 ...ult)
EN	Unknown	PIN_AB18	4	B4_N0	2.5 ...ult)
clk	Unknown	PIN_AB15	4	B4_N1	2.5 ...ult)
Q1	Unknown	PIN_U12	4	B4_N1	2.5 ...ult)
Q2	Unknown	PIN_V12	4	B4_N1	2.5 ...ult)

6. 实验现象

Q1: 先设置 D 的数值，按 clk 按钮的时候观察 Q1 的亮灭情况。当松开按钮前 D 为 1 时，小灯泡会灭，证实了程序正确。

Q2: 先设置 D 的数值，然后调整 EN 的数值，当 EN 为 1，小灯泡随着 D 数值的改变而改变亮灭，一旦 EN 设置为 0，则无论 D 的改变，小灯泡的亮灭情况不会改变。

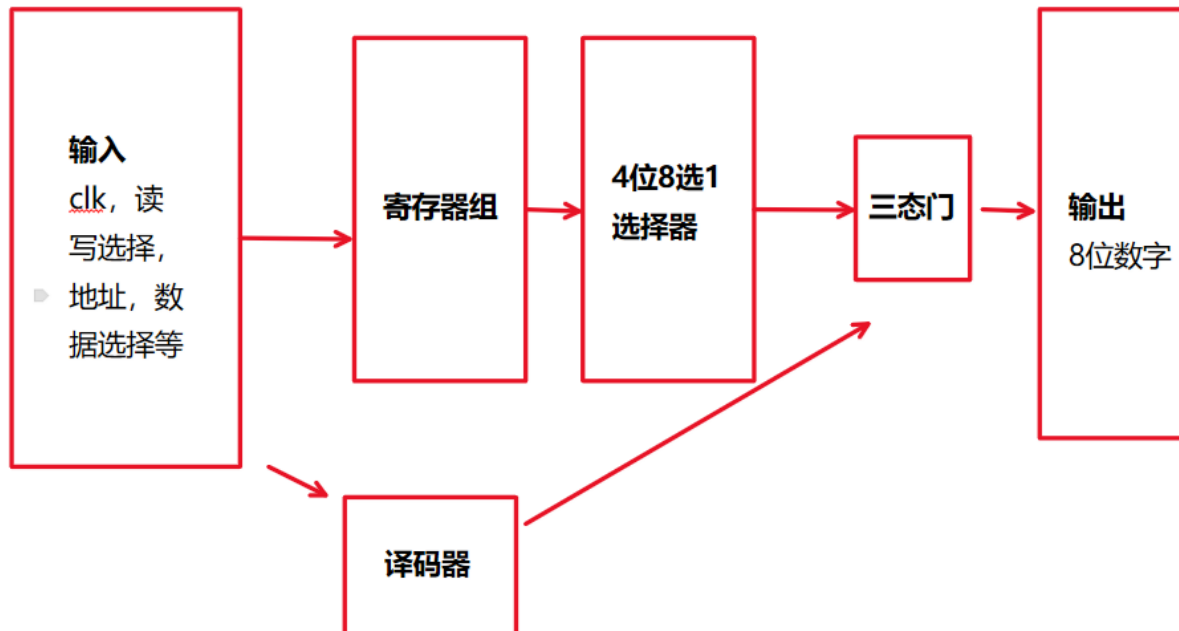
三、任务二设计与实现

1. 要求

- (1) 设计一个由 8 个 32 位寄存器构成的寄存器堆，该寄存器堆支持寄存器地址选择，能够写入任意值，并且可从任意一个寄存器中读取数据。
- (2) 读操作不需要时钟控制，写操作需要在上升沿写入。
- (3) 用地址译码、寄存器组、数据选择电路等基本电路实现。

2. 设计思路

分析寄存器堆的组成，发现他由四个模块组成。分别是译码器，数据选择器，寄存器组和三态门。分别将寄存器和与门等用 Verilog 语言写出并封装，再将其合理连接至一起，组成寄存器组，再与数据选择器等联结，形成最终电路。



3. 详细设计

以下分别为 3-8 译码器，寄存器（单个），选择器，三态门的 Verilog 代码，以及寄存器组、寄存器堆的电路实现。

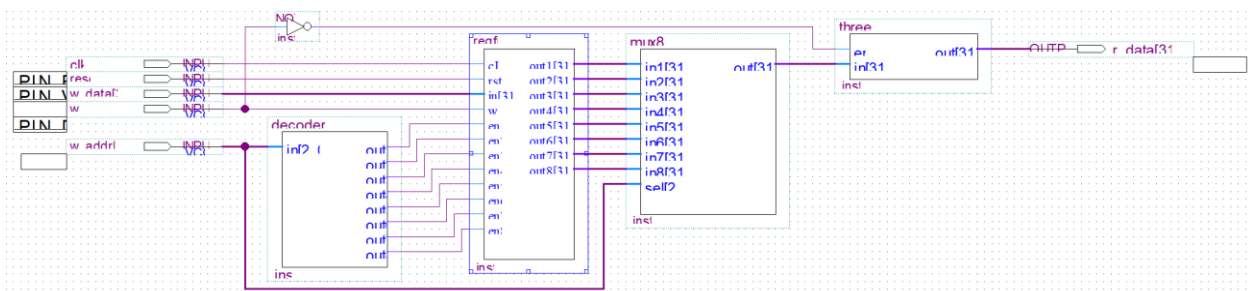
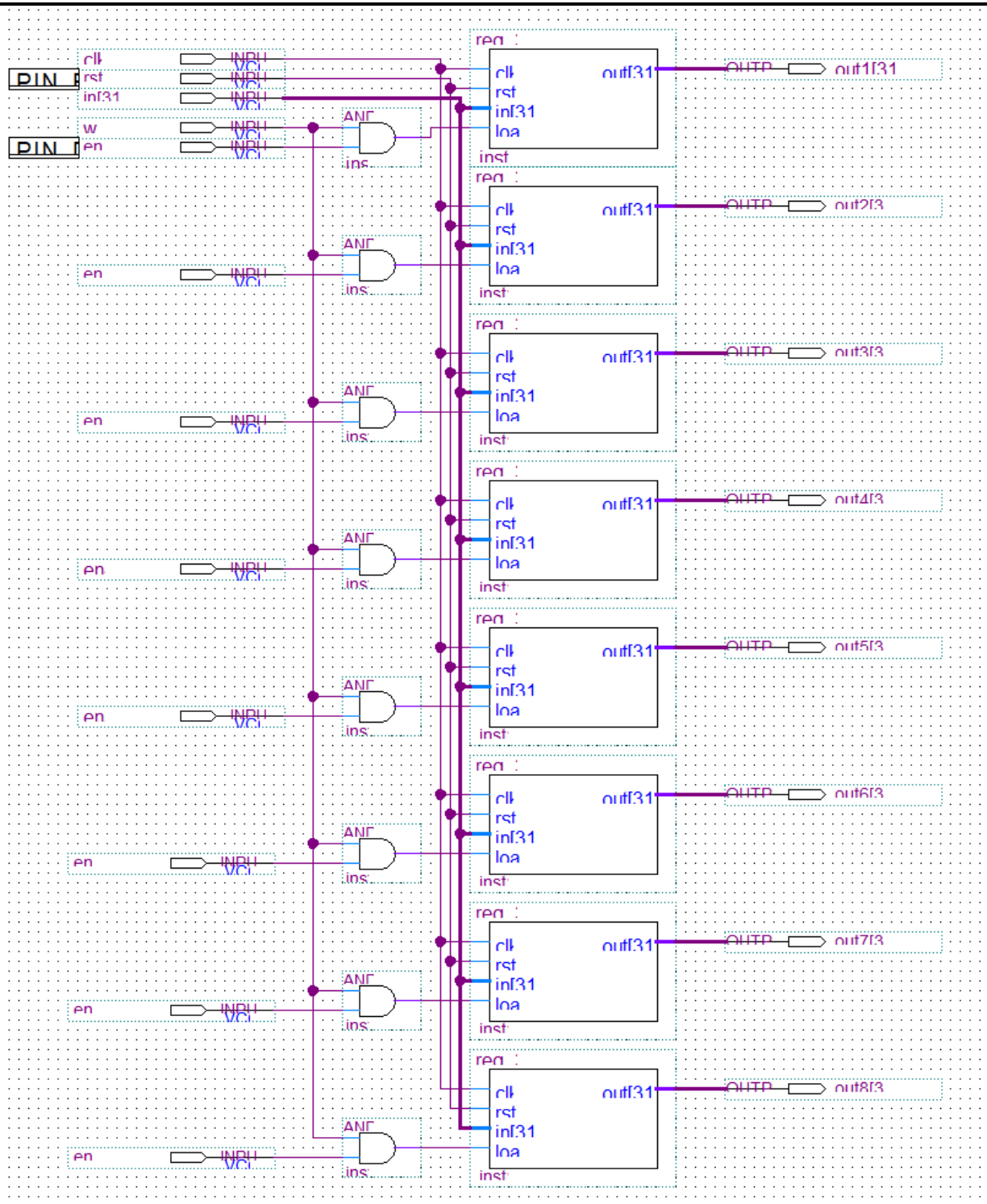
```

1 module decoder3_8(in, out1, out2, out3, out4, out5, out6, out7, out8);
2     input [2:0] in;
3     output out1, out2, out3, out4, out5, out6, out7, out8;
4     reg out1, out2, out3, out4, out5, out6, out7, out8;
5     always @ (in)
6     begin
7         case(in)
8             3'b000: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b10000000;
9             3'b001: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b01000000;
10            3'b010: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00100000;
11            3'b011: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00010000;
12            3'b100: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00001000;
13            3'b101: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00000100;
14            3'b110: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00000010;
15            3'b111: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00000001;
16            default: {out1, out2, out3, out4, out5, out6, out7, out8} = 8'b00000000;
17        endcase
18    end
19 endmodule

1 module reg_32(clk, rst_n, in, load, out);
2     input clk, rst_n, load;
3     input [31:0] in;
4     output [31:0] out;
5     reg [31:0] out;
6     always @ (posedge clk, negedge rst_n)
7     if (~rst_n)
8         out <= 0;
9     else if (load == 1)
10        out <= in;
11 endmodule
    
```

```
1 module mux8_1(in1, in2, in3, in4, in5, in6, in7, in8, sel, out);
2     input [2:0] sel;
3     input [31:0] in1, in2, in3, in4, in5, in6, in7, in8;
4     output [31:0] out;
5     reg [31:0] out;
6     always @ (sel)
7     begin
8         case(sel)
9             3'b000: out = in1;
10            3'b001: out = in2;
11            3'b010: out = in3;
12            3'b011: out = in4;
13            3'b100: out = in5;
14            3'b101: out = in6;
15            3'b110: out = in7;
16            3'b111: out = in8;
17            default: out = 32'b0;
18        endcase
19    end
20 endmodule

1 module three_t(en, in, out);
2     input en;
3     input [31:0] in;
4     output [31:0] out;
5     assign out = (en == 1) ? in : 32'bz;
6 endmodule
```

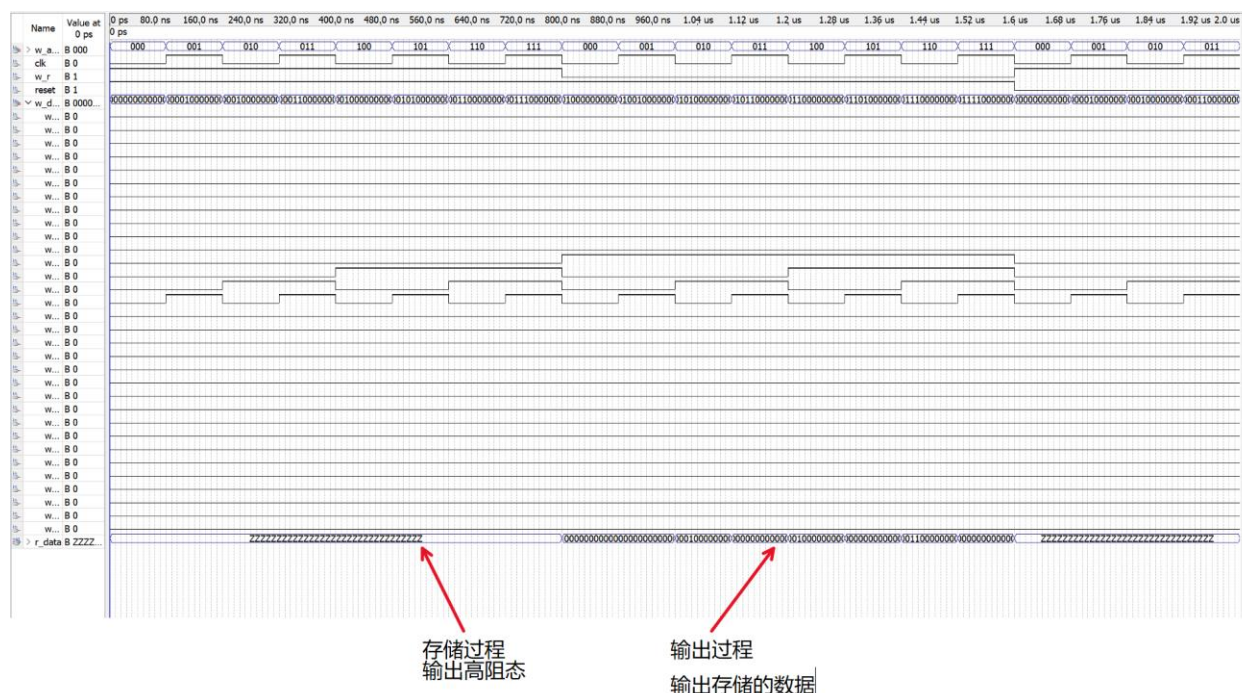


4. 仿真验证

数字逻辑实验报告

实验 1

如下图，请放大以看清楚，为最终寄存器堆的仿真图表。初始 w_r 为 1 时，此装置属于存储状态，存入 4 位测试值。输出时，根据选择器的值，输出特定的 32 位（含测试用的 4 位值，其他数位为默认值 0）。



5. 引脚分配

如图，下面为已设置引脚的输入/输出值。其他的因实验器材输出范围问题，未设置引脚。输入部分，除 clk 使用按钮输入以外，其他输入引脚选择了开关。输出设置为 LED 灯引脚。

out	r_data[7]	Output	PIN_U12	4	B4_N1	PIN_U12	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[6]	Output	PIN_V12	4	B4_N1	PIN_V12	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[5]	Output	PIN_V15	4	B4_N0	PIN_V15	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[4]	Output	PIN_W13	4	B4_N1	PIN_W13	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[3]	Output	PIN_W15	4	B4_N0	PIN_W15	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[2]	Output	PIN_Y17	4	B4_N0	PIN_Y17	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[1]	Output	PIN_R16	4	B4_N0	PIN_R16	2.5 ...ult)	8mA ...ult)	2 (default)	
out	r_data[0]	Output	PIN_T17	5	B5_N1	PIN_T17	2.5 ...ult)	8mA ...ult)	2 (default)	
in	reset	Input	PIN_V13	4	B4_N1	PIN_V13	2.5 ...ult)	8mA ...ult)		
in	w_addr[2]	Input	PIN_C8	8	B8_N0	PIN_C8	2.5 ...ult)	8mA ...ult)		
in	w_addr[1]	Input	PIN_E7	8	B8_N1	PIN_E7	2.5 ...ult)	8mA ...ult)		
in	w_addr[0]	Input	PIN_F8	8	B8_N1	PIN_F8	2.5 ...ult)	8mA ...ult)		
in	w_data[7]	Input	PIN_AB17	4	B4_N0	PIN_AB17	2.5 ...ult)	8mA ...ult)		
in	w_data[6]	Input	PIN_AB18	4	B4_N0	PIN_AB18	2.5 ...ult)	8mA ...ult)		
in	w_data[5]	Input	PIN_C3	8	B8_N1	PIN_C3	2.5 ...ult)	8mA ...ult)		
in	w_data[4]	Input	PIN_E5	8	B8_N1	PIN_E5	2.5 ...ult)	8mA ...ult)		
in	w_data[3]	Input	PIN_C7	8	B8_N0	PIN_C7	2.5 ...ult)	8mA ...ult)		
in	w_data[2]	Input	PIN_E6	8	B8_N1	PIN_E6	2.5 ...ult)	8mA ...ult)		
in	w_data[1]	Input	PIN_F7	8	B8_N1	PIN_F7	2.5 ...ult)	8mA ...ult)		
in	w_data[0]	Input	PIN_A3	8	B8_N1	PIN_A3	2.5 ...ult)	8mA ...ult)		
in	w_r	Input	PIN_D6	8	B8_N1	PIN_D6	2.5 ...ult)	8mA ...ult)		

6. 实验现象

通过调节选择端、输入数据端对应的开关，以及点按 clk 按钮，8 片寄存器每一片存入了我们想要的数字。随后将 w_r 拨至 0，使其处于输出（读取）模式，调节选择端，以读取之前存入的数据，LED 显示 8 位对应的数据。拨动 $reset$ ，LED 全亮，代表数据已清空，输出置为全 0。

四、总结

数字逻辑实验报告

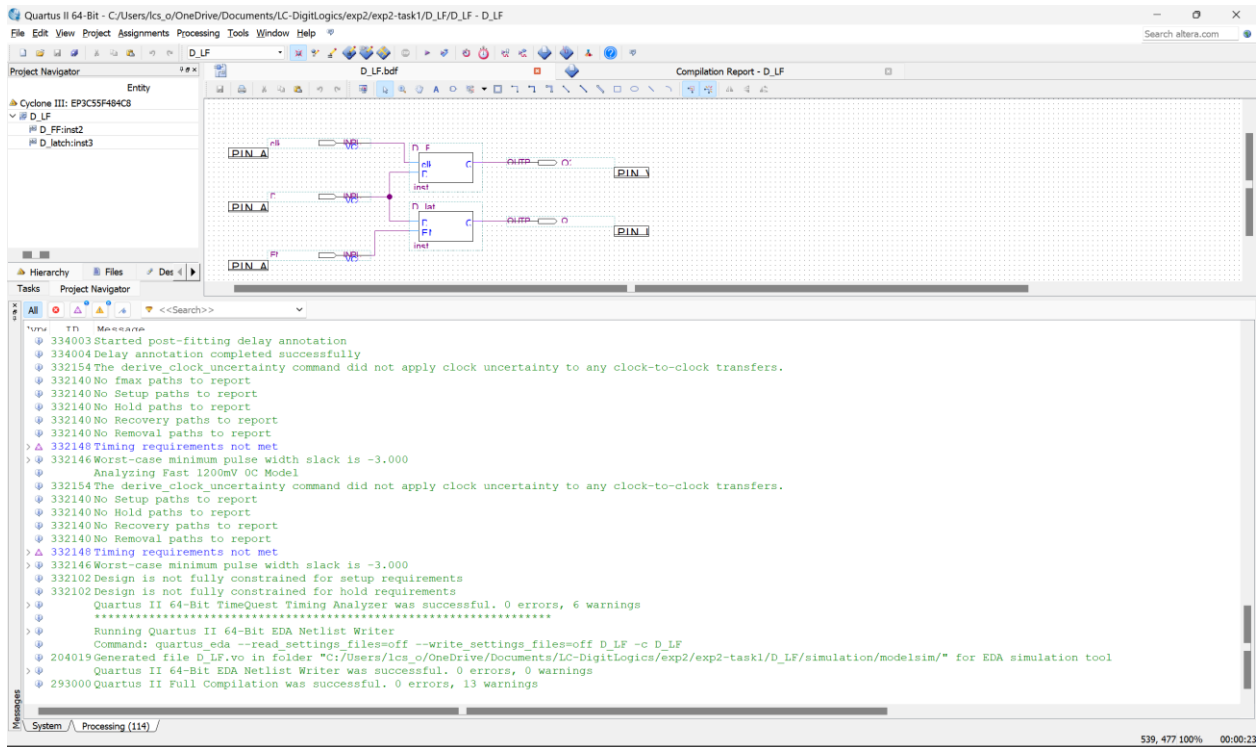
实验 1

我学会了 testbench 的编写、仿真方法。

遇到的问题：误认为 clk 为按动并持续保持值（而并非其固有的按动触发的响应方式）；使用 Testbench 时遇到了配置安装环境的问题；

这些问题均已通过学习的方式解决。

附图 1：任务一顶层及综合通过图



附图 2：任务二顶层及综合通过图

